

PROYECTO FINAL

Informe

Técnicas de Programación Orientada a Objetos · Ingeniería de Sistemas · 5to Ciclo · 2026

Grupo:	Grupo 15
Integrante 1	Jonathan Calderón Gamarra (N00571575)
Integrante 2	Karla Mariel Quispe Padilla (N00353037)
Integrante 3	Andre Aaron Colchado Robles (N00418772)
Integrante 4	Dara Rina Borja Goñi (N00422554)
Empresa / caso	IronForge Gym
Docente	Victor Murguerza

 Instrucciones generales

Este formato guía la elaboración del informe del Proyecto Final. Completa cada sección con los datos reales de tu empresa o caso ficticio. Lee los recuadros de ayuda (💡 **Hint**) antes de desarrollar cada punto — contienen orientación para evitar los errores más comunes.

- **Entrega:** un archivo .docx con este informe + un archivo .zip con los archivos .java del proyecto.
- El código debe compilar y ejecutarse sin errores en VSCode con Java 17+.
- El diagrama UML debe ser una imagen insertada en el informe (captura de StarUML, draw.io o similar).
- La empresa puede ser real (donde trabajas o conoces) o ficticia, pero el problema debe ser auténtico.

01 Identificar y formular el problema

2 pts

1.1 Descripción de la empresa o contexto

Nombre: IronForge Fitness Club S.A.C.

Nombre comercial: IronForge Gym

Rubro: Servicios deportivos y acondicionamiento físico.

Ubicación: Yanahuara, Arequipa, Perú.

Tamaño: Aproximadamente 7 empleados y 420 clientes registrados, de los cuales alrededor de 280 mantienen una membresía activa.

Área donde se origina el problema: Recepción y administración.

IronForge Gym es un gimnasio dedicado a brindar servicios de entrenamiento físico y acondicionamiento muscular. El área de recepción se encarga del registro de nuevos clientes, gestión de membresías y control de ingreso al establecimiento.

1.2 Descripción del problema actual

Actualmente, el registro de clientes y el control de ingreso se realizan de manera manual mediante cuadernos físicos. Los datos personales de los clientes, las membresías y los ingresos son registrados por el personal de recepción.

Este proceso dificulta la búsqueda y actualización de información. Además, verificar manualmente si un cliente tiene una membresía vigente puede generar demoras y errores en el control de acceso.

En horas de mayor afluencia, la búsqueda de información puede tomar aproximadamente entre 2 y 4 minutos por cliente. Asimismo, los registros físicos pueden deteriorarse, extraviarse o presentar información incompleta.

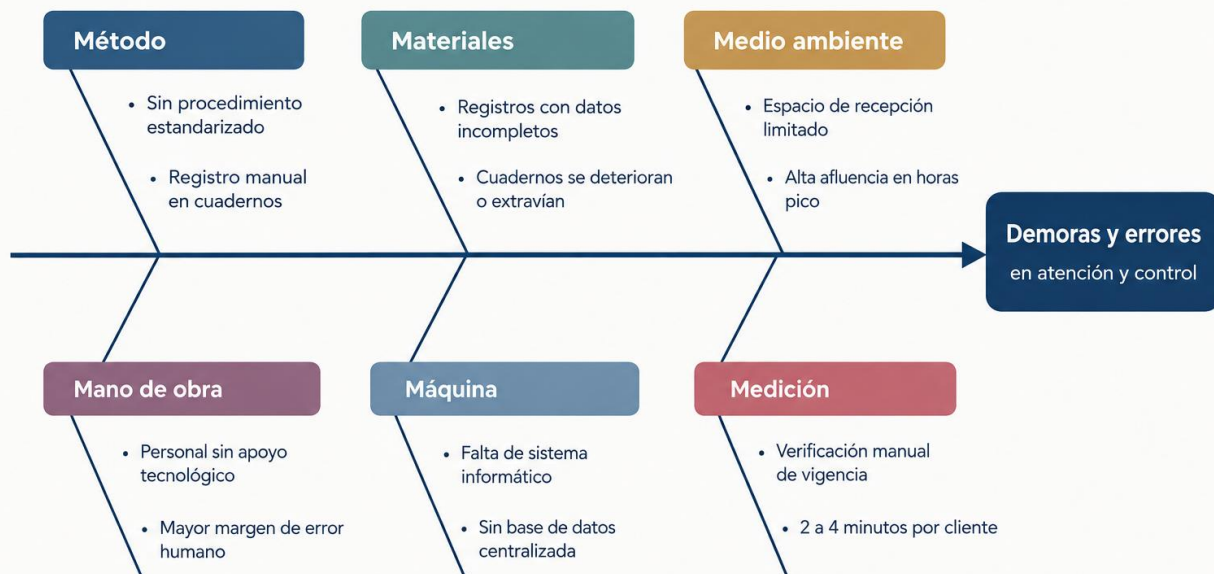
Como consecuencia, el gimnasio presenta demoras en la atención, dificultad para consultar el historial de ingresos y mayor riesgo de errores en el control de las membresías.

Diagrama de Ishikawa

En este apartado se colocará el diagrama de Ishikawa correspondiente al problema identificado: registro manual de clientes y control manual de ingreso en IronForge Gym.

Diagrama de Ishikawa

Registro manual de clientes y control de ingreso



1.3 Formulación del problema (pregunta)

¿Cómo automatizar el registro de clientes y el control de ingreso en IronForge Gym mediante un sistema de escritorio desarrollado en Java y conectado a una base de datos MySQL, con la finalidad de mejorar la gestión de la información y reducir las demoras en la atención?

02 Antecedentes del problema identificado

2 pts

2.1 Antecedente 1

Castañeda Pisani y Guillen Vilchez (2023) desarrollaron una propuesta de implementación de una página web para el control de clientes y asistencia en la empresa Susan Gym. El estudio identificó dificultades en el manejo manual de los datos de los clientes y planteó una solución web que permitiera registrar, consultar y controlar la información de manera más rápida y organizada. Los autores concluyeron que la digitalización de estos procesos puede facilitar la gestión de clientes y mejorar el control de la asistencia en el gimnasio.

Cita: (Castañeda Pisani & Guillen Vilchez, 2023)

Fuente: Universidad César Vallejo – Facultad de Ingeniería, Escuela Profesional de Ingeniería de Sistemas. Documento disponible públicamente en Studocu

2.2 Antecedente 2

Rodríguez Portilla (2024) – Pontificia Universidad Católica del Ecuador

El proyecto *Sistema web para la gestión de accesos al club de fortalecimiento muscular de la PUCE-I mediante un decodificador de credenciales* abordó dificultades relacionadas con el registro y control de accesos de un gimnasio.

El problema estaba relacionado con el uso de procedimientos manuales y hojas de cálculo para administrar los accesos, lo que podía generar errores en el registro y afectar la integridad de la información.

Como solución se planteó un sistema web para optimizar el registro y control de los accesos al gimnasio, permitiendo gestionar la información de manera digital.

Este antecedente se relaciona directamente con el proyecto propuesto para IronForge Gym, debido a que ambos buscan mejorar el control de acceso mediante la digitalización del proceso.

Cita: (Rodríguez Portilla, 2024)

Fuente: Pontificia Universidad Católica del Ecuador, Repositorio Institucional.

03 Establecer los objetivos del proyecto

2 pts

3.1 Objetivo general

Desarrollar, en un periodo de 4 semanas, un sistema de escritorio para la gestión de clientes y el control de ingreso de IronForge Gym, utilizando Java, JavaFX y MySQL, que permita digitalizar los procesos de registro de clientes y control de acceso, reduciendo a menos de 1 minuto el tiempo de registro de cada ingreso en comparación con el proceso manual actual realizado mediante cuadernos físicos.

3.2 Objetivo específico 1

Analizar y documentar los requerimientos funcionales del sistema mediante historias de usuario y criterios de aceptación verificables.

3.3 Objetivo específico 2

Diseñar un diagrama de clases UML que represente las entidades, relaciones y principales componentes del sistema.

3.4 Objetivo específico 3

Implementar el sistema en Java, utilizando clases del modelo, controladores, servicios y DAO para gestionar la información en MySQL mediante JDBC. En la primera etapa, las operaciones se demostrarán mediante programas sencillos de consola; posteriormente, se incorporará una interfaz gráfica en JavaFX.

3.5 Objetivo específico 4

Validar el funcionamiento del sistema mediante pruebas de los principales requerimientos funcionales y sus criterios de aceptación.

4

Alcance de la solución

2 pts

4.1 ¿Qué incluye el sistema? (dentro del alcance)

El sistema incluirá los siguientes módulos y funcionalidades:

- Gestión de clientes.
- Registro, consulta y actualización de clientes.
- Gestión de tipos de membresía.
- Registro y consulta de membresías.
- Verificación de vigencia de membresías.
- Registro de ingresos de clientes.
- Consulta del historial de ingresos.
- Búsqueda de clientes por DNI.
- Consulta de membresías próximas a vencer.
- Gestión básica de usuarios del sistema.
- Inicio de sesión y control de acceso según rol.
- Almacenamiento de información en una base de datos MySQL.
- La entrega del T2 y PC03 funcionan por consola y posteriormente se incorporará en JavaFX (Reutilizando los servicios y DAO)

4.2 ¿Qué NO incluye el sistema? (fuera del alcance)

El sistema no incluirá:

- Aplicación móvil.
- Página web.
- Pagos electrónicos.
- Integración con bancos o billeteras digitales.
- Facturación electrónica.
- Integración con SUNAT.
- Reconocimiento facial.
- Control biométrico.
- Lectores de huella digital.
- Torniquetes automáticos.
- Gestión de rutinas de entrenamiento.
- Gestión nutricional.
- Integración con WhatsApp.
- Inteligencia artificial.

05 Requerimientos Funcionales (mínimo 10)

2 pts

Código	Módulo	Descripción del requerimiento	Prior.
--------	--------	-------------------------------	--------

RF-01	Clientes	El sistema debe permitir al recepcionista registrar clientes con DNI, nombres y apellidos obligatorios y teléfono opcional, impidiendo registros con datos obligatorios vacíos o DNI duplicado.	Alta
RF-02	Clientes	El sistema debe permitir al recepcionista buscar un cliente por su DNI y mostrar sus datos personales, o indicar que no existe un cliente registrado con ese documento.	Alta
RF-03	Clientes	El sistema debe permitir al recepcionista actualizar los nombres, apellidos y teléfono de un cliente registrado, validando los campos obligatorios y conservando sus membresías e ingresos asociados.	Alta
RF-04	Membresía	El sistema debe permitir al recepcionista consultar la vigencia de las membresías de un cliente. Una membresía estará vigente cuando la fecha de consulta se encuentre entre sus fechas de inicio y vencimiento, ambas incluidas; fuera de ese período, se mostrará como aún no vigente o vencida, según corresponda.	Alta
RF-05	Membresía	El sistema debe permitir al recepcionista registrar una nueva membresía para un cliente existente, indicando un tipo de membresía registrado, la fecha de inicio y la fecha de vencimiento. Ambas fechas son obligatorias y la fecha de vencimiento no puede ser anterior a la fecha de inicio.	Alta
RF-06	Ingresos	El sistema debe permitir al recepcionista registrar el ingreso de un cliente mediante su DNI, asociando el registro al cliente y a la membresía que autoriza el acceso.	Alta
RF-07	Ingresos	El sistema debe verificar que el cliente exista y cuente con una membresía vigente antes de autorizar un ingreso. Si alguna condición no se cumple, debe informar el motivo y no crear el registro de ingreso.	Alta
RF-08	Ingresos	El sistema debe guardar automáticamente la fecha y hora de cada ingreso autorizado, obtenidas del reloj del sistema.	Alta
RF-09	Consultas	El sistema debe permitir al administrador consultar el historial de ingresos de un cliente mediante su	Media

		DNI, mostrando la fecha y hora de cada registro e indicando cuando no existan ingresos.	
RF-10	Usuarios	El sistema debe permitir a los usuarios activos iniciar sesión mediante su nombre de usuario y contraseña. Si las credenciales son incorrectas o la cuenta está inactiva, debe rechazar el acceso.	Media
RF-11	Consultas	El sistema debe permitir al administrador consultar las membresías vigentes que vencen entre la fecha actual y los siguientes siete días, ambos extremos incluidos, mostrando el nombre y DNI del cliente y la fecha de vencimiento.	Alta
RF-12	Membresías	El sistema debe permitir al recepcionista renovar la membresía de un cliente mediante el registro de un nuevo período, conservando las membresías anteriores y aplicando las validaciones de fechas establecidas en el RF-05.	Alta
RF-13	Usuarios	. El sistema debe permitir al administrador crear usuarios del personal con nombre de usuario único, contraseña y rol Administrador o Recepcionista, validando que esos datos sean obligatorios. Las cuentas nuevas se registrarán como activas.	Alta
RF-14	Usuarios	El sistema debe permitir al administrador activar o desactivar las cuentas del personal, impidiendo el inicio de sesión de los usuarios inactivos.	Alta
RF-15	Membresías	El sistema debe permitir al administrador registrar y consultar tipos de membresía con un nombre obligatorio y único, y permitir al recepcionista consultar los tipos disponibles al registrar una membresía.	Alta

RF-16	Usuarios	El sistema debe restringir las operaciones según el rol del usuario autenticado. El Recepcionista podrá registrar, buscar y actualizar clientes; consultar los tipos de membresía disponibles; registrar y renovar membresías; consultar su vigencia y registrar ingresos. El Administrador podrá consultar historiales de ingresos y membresías próximas a vencer, registrar y consultar tipos de membresía, y crear, activar y desactivar cuentas del personal.	Alta
--------------	----------	---	------

06 Historias de Usuario con Criterios de Aceptación

2 pts

Completa las siguientes historias de usuario (mínimo 5, máximo 10):

HU-01	Título de la historia: Registro de clientes
Como	Recepcionista
Quiero	Registra los datos de nuevos clientes
Para	Mantener organizada la información de los clientes y evitar registros manuales
Criterios de aceptación	✓ CA-01: Dado que no existe un cliente con DNI 12345678, cuando el recepcionista registra a Ana Torres con ese DNI y teléfono 987654321, entonces el sistema guarda al cliente, muestra el mensaje «Cliente registrado correctamente» y permite consultar los datos registrados mediante su DNI. ✓ CA-02: Dado que el recepcionista ingresa el DNI 12345678 y deja vacíos los nombres y apellidos, cuando intenta guardar al cliente, entonces el sistema impide el registro y muestra el mensaje «Los nombres y apellidos son obligatorios»

HU-02	Título de la historia: Consulta de clientes
Como	repcionista
Quiero	buscar clientes mediante su DNI
Para	consultar rápidamente su información.
Criterios de aceptación	✓ CA-01: Cliente encontrado: Dado que Ana Torres López está registrada con DNI 12345678 y teléfono 987654321, cuando el recepcionista busca ese DNI, entonces el sistema muestra sus nombres, apellidos, DNI y teléfono. ✓ CA-2: Dado que no existe un cliente con DNI 87654321, cuando el recepcionista busca ese documento, entonces el sistema muestra «Cliente no encontrado» y no presenta datos de otro cliente.

HU-03	Título de la historia: Gestión de membresías
Como	repcionista
Quiero	registrar una membresía para un cliente
Para	mantener actualizado su período de acceso al gimnasio
Criterios de aceptación	✓ CA-01: Dado que Existe un cliente registrado, cuando El recepcionista selecciona un tipo de membresía e ingresa las fechas correspondientes, entonces El sistema debe registrar la membresía asociada al cliente. ✓ CA-02: Dado que No existe el cliente seleccionado, cuando El recepcionista intenta registrar la membresía, entonces El sistema debe impedir la operación y mostrar un mensaje de error.

HU-04	Título de la historia: Verificación de membresía
Como	repcionista.
Quiero	consultar la vigencia de la membresía de un cliente.
Para	determinar si puede ingresar al gimnasio.
Criterios de aceptación	✓ CA-01: Dado que El cliente tiene una membresía vigente, cuando El recepcionista consulta su membresía, entonces El sistema debe indicar que la membresía se encuentra vigente.

✓ **CA-02:** Dado que El cliente tiene una membresía vencida, cuando El recepcionista consulta su membresía, entonces El sistema debe indicar que la membresía se encuentra vencida.

HU-05	Título de la historia: Control de ingreso
Como	recepcionista
Quiero	registrar el ingreso de los clientes al gimnasio
Para	controlar los accesos y mantener un historial digital.
Criterios de aceptación	✓ CA-01: Dado que el cliente se encuentra registrado y posee una membresía vigente, cuando el recepcionista registra su ingreso al gimnasio, entonces el sistema registra el ingreso y almacena automáticamente la fecha y hora. ✓ CA-02: Dado que El cliente tiene una membresía vencida, cuando El recepcionista intenta registrar su ingreso, entonces El sistema debe rechazar el ingreso y mostrar el motivo.

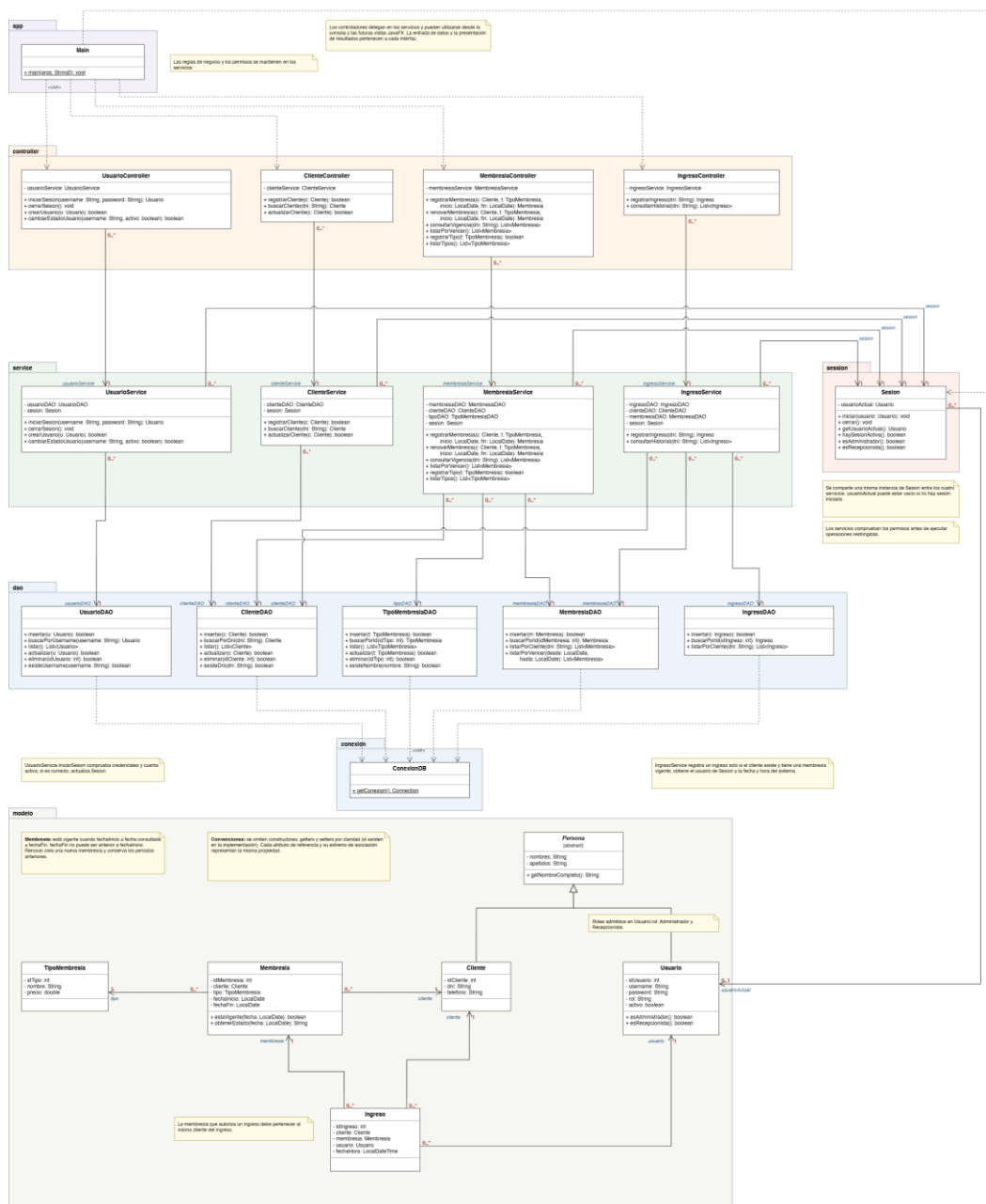
HU-06	Título de la historia: Consulta de historial de ingresos
Como	administrador.
Quiero	consultar el historial de ingresos.
Para	conocer los accesos registrados de los clientes.
Criterios de aceptación	✓ CA-01: Dado que Existen registros de ingreso, cuando El administrador consulta el historial, entonces El sistema debe mostrar los ingresos registrados. ✓ CA-02: Dado que No existen registros para la consulta realizada, cuando El administrador realiza la búsqueda, entonces El sistema debe indicar que no existen registros.

HU-07 Como Quiero Para	Título de la historia: Consulta de membresías por vencer administrador. consultar las membresías próximas a vencer. identificar a los clientes que deben renovar su membresía.
Criterios de aceptación	✓ CA-01: Dado que Existen membresías próximas a vencer, cuando El administrador realiza la consulta, entonces El sistema debe mostrar los clientes correspondientes. ✓ CA-02: Dado que No existen membresías próximas a vencer, cuando El administrador realiza la consulta, entonces El sistema debe mostrar un mensaje indicando que no existen resultados.
HU-08 Como Quiero Para	Título de la historia: Inicio de sesión usuario autorizado. iniciar sesión con mi usuario y contraseña. acceder de manera segura a las funciones del sistema.
Criterios de aceptación	✓ CA-01: Dado que El usuario está registrado y sus credenciales son correctas., cuando Ingresa su usuario y contraseña, entonces El sistema debe permitir el acceso. ✓ CA-02: Dado que El usuario ingresa credenciales incorrectas, cuando Intenta iniciar sesión., entonces El sistema debe rechazar el acceso y mostrar un mensaje de error.
HU-09 Como Quiero Para	Título de la historia: Renovación de membresía recepcionista. renovar la membresía de un cliente mantener vigente su acceso al gimnasio conservando sus períodos anteriores.
Criterios de aceptación	✓ CA-01: Dado que el cliente posee una membresía registrada, cuando el recepcionista registra una renovación con fechas válidas, entonces el sistema crea un nuevo

	período de membresía asociado al cliente y conserva las membresías anteriores.. ✓ CA-02: Dado que las fechas ingresadas para la renovación no son válidas, cuando el recepcionista intenta registrar el nuevo período, entonces el sistema impide la operación y muestra un mensaje de error.
--	--

HU-10	Título de la historia: Gestión de usuarios y roles
Como	administrador
Quiero	crear y administrar las cuentas del personal
Para	controlar quién puede acceder al sistema y qué funciones puede utilizar.
Criterios de aceptación	✓ CA-01: Dado que el administrador ingresa un nombre de usuario único, contraseña y rol válido, cuando registra al nuevo usuario, entonces el sistema crea la cuenta en estado activo. ✓ CA-02: Dado que existe una cuenta de usuario registrada, cuando el administrador la desactiva, entonces dicho usuario no puede iniciar sesión en el sistema

⚠ El criterio de aceptación NO es una lista de lo que hace el sistema. Es una condición verificable: se puede probar que se cumple o no se cumple.



<https://drive.google.com/file/d/1GFdkwf6aa3a3aS16jS6SdVrUL2M3idW4/view?usp=sharing>

"Todas las clases del modelo tienen atributos privados con sus métodos get y set públicos; se omiten en el diagrama por claridad."

Nota sobre la organización de paquetes

En la implementación Java, las clases del modelo se ubicarán en el paquete `com.tpoo.upn.model` y la clase `ConexionDB` se conservará en `com.tpoo.upn.dao`. Por tanto, el bloque denominado «modelo» en el diagrama corresponde al paquete `model`; el bloque «conexion» agrupa visualmente a `ConexionDB`, pero no representa un paquete independiente en el código.

Esta diferencia de organización no modifica las responsabilidades ni las relaciones entre las clases: los controladores utilizan los servicios, los servicios utilizan los DAO y los DAO obtienen la conexión mediante `ConexionDB`.

7.1 Justificación de las relaciones más importantes

Herencia: Cliente y Usuario heredan de Persona

La clase abstracta `Persona` reúne los atributos comunes, como nombres y apellidos. `Cliente` y `Usuario` heredan estos atributos y agregan sus propios datos. `Cliente` incorpora DNI y teléfono, mientras que `Usuario` incorpora nombre de usuario, contraseña, rol y estado de la cuenta.

Se utiliza herencia porque tanto un cliente como un usuario del personal son personas. `Persona` es abstracta porque no se crearán objetos directamente de esta clase, sino objetos de `Cliente` o `Usuario`.

Asociación: Cliente 1 — 0..* Membresia

Cada membresía pertenece a un único cliente. Un cliente puede no tener membresías o contar con varias registradas a lo largo del tiempo, debido a sus renovaciones.

La clase `Membresia` mantiene una referencia al cliente al que pertenece. Para consultar sus membresías, `MembresiaService` utiliza `MembresiaDAO`. Las renovaciones crean nuevos períodos y conservan los anteriores.

Asociación: TipoMembresia 1 — 0..* Membresia

Cada membresía corresponde a un único tipo, por ejemplo, mensual o trimestral. Un mismo tipo puede estar asociado con varias membresías o todavía no haber sido utilizado.

La clase `Membresia` mantiene una referencia a `TipoMembresia`. Esta relación permite reutilizar los tipos registrados sin repetir su información en cada membresía.

Asociación: Cliente 1 — 0..* Ingreso

Cada ingreso corresponde a un único cliente. Un cliente puede no tener ingresos registrados o contar con varios.

Esta relación permite identificar quién ingresó al gimnasio y consultar su historial de accesos.

Asociación: Membresia 1 — 0..* Ingreso

Cada ingreso autorizado se relaciona con una única membresía. Una membresía puede autorizar varios ingresos durante su período de vigencia o no tener ingresos registrados.

Antes de guardar un ingreso, IngresoService comprueba que la membresía esté vigente y pertenezca al mismo cliente asociado al registro.

Asociación: Usuario 1 — 0..* Ingreso

Cada ingreso es registrado por un único usuario del sistema con el permiso correspondiente. Un usuario puede registrar varios ingresos o no haber registrado ninguno.

Esta relación permite identificar al miembro del personal que realizó cada registro de acceso.

Relación entre la presentación, los controladores, los servicios y los DAO

Los programas de consola recibirán los datos y mostrarán los resultados. Para realizar cada operación, llamarán al controlador correspondiente: ClienteController, UsuarioController, MembresiaController o IngresoController.

Cada controlador mantendrá una referencia a su servicio correspondiente y delegará en él las operaciones solicitadas. Por ejemplo, ClienteController utilizará ClienteService para registrar, buscar y actualizar clientes. Los controladores no ejecutarán consultas SQL ni duplicarán las reglas de negocio.

Los servicios comprobarán los permisos y las reglas del sistema. ClienteService verificará que los datos obligatorios estén completos y que el DNI no esté registrado. IngresoService comprobará que el cliente exista y tenga una membresía vigente antes de registrar su ingreso.

Los servicios utilizarán los DAO para consultar y guardar información en MySQL. Las llamadas seguirán la estructura: presentación → controlador → servicio → DAO.

Los controladores se mantendrán independientes de la lectura por teclado y de los elementos gráficos. En la siguiente etapa, las vistas de JavaFX utilizarán estos controladores, conservando los servicios y DAO.

Dependencia: DAO → ConexionDB

Las clases DAO utilizarán ConexionDB para obtener una conexión con la base de datos MySQL cuando necesiten realizar una operación.

ConexionDB centralizará la configuración y el método para obtener la conexión. Esta relación se representará mediante una dependencia, porque los DAO utilizan esta clase como apoyo para acceder a la base de datos.

Uso de Sesion y validación de permisos

La clase Sesion mantendrá una referencia al usuario que haya iniciado sesión. En cada ejecución se compartirá una misma instancia de Sesion entre los servicios utilizados.

UsuarioController delegará el inicio y cierre de sesión en UsuarioService. Los demás controladores también delegarán sus operaciones en sus respectivos servicios.

Los servicios comprobarán la sesión y el rol antes de ejecutar operaciones restringidas. Por ejemplo, la creación, activación y desactivación de cuentas corresponderán al administrador.

Los programas de consola mostrarán los resultados o los mensajes de error. En la siguiente etapa, las vistas de JavaFX presentarán las opciones correspondientes al rol del usuario. La validación de permisos permanecerá en los servicios.

Estado calculado de la membresía

El estado de la membresía se calculará a partir de sus fechas de inicio y vencimiento y de la fecha consultada. No se almacenará como un dato permanente en la base de datos.

La membresía se considerará VIGENTE cuando la fecha consultada esté entre el inicio y el vencimiento, incluyendo ambas fechas; AÚN NO VIGENTE cuando sea anterior al inicio; y VENCIDA cuando sea posterior al vencimiento.

La clase Membresia podrá realizar esta comprobación mediante un método que los servicios utilizarán al consultar la vigencia o autorizar un ingreso.

Uso de asociaciones simples

Las relaciones entre las entidades se representarán mediante asociaciones simples para mostrar sus vínculos y multiplicidades.

Por ejemplo, una membresía debe estar vinculada a un cliente, pero esta condición no implica que deba representarse mediante composición. El sistema necesita conservar los períodos de membresía y los registros de ingreso para consultar el historial.

No se incorporarán relaciones de agregación o composición sin una necesidad concreta. Las relaciones del diagrama deberán corresponder con las responsabilidades y referencias que se implementen en el código.

08 Implementación del Sistema en Java4 pts — entrega
código en .zip**💡 ¿Qué debe incluir el código de implementación?**

- Las clases del modelo (entidades) con encapsulamiento completo: atributos private, getters, setters con validación y constructores.
- Una clase DAO por cada entidad principal que maneje las operaciones de BD: insertar(), buscar(), listar(), actualizar(), eliminar().
- La interfaz gráfica en JavaFX o Swing conectada al modelo mediante el patrón MVC básico: la vista no debe contener lógica de negocio.
- Una clase de conexión centralizada (ConexionDB) y el script SQL para crear la base de datos.
- Un método main() que demuestre al menos 5 RF funcionando con datos reales.

☑ Checklist de entrega — verifica antes de enviar

- ☐ El código compila y ejecuta sin errores desde VSCode con Java 17
- ☐ Mínimo 5 clases del modelo con encapsulamiento correcto (private + getters/setters)
- ☐ Mínimo 3 clases DAO con operaciones CRUD usando PreparedStatement
- ☐ Interfaz gráfica funcional con al menos 3 ventanas o formularios
- ☐ Base de datos MySQL con tablas relacionadas (mínimo 3 tablas con FK)
- ☐ Los 10 RF están implementados y son demostrables en la interfaz
- ☐ No hay SQL hardcodeado fuera del DAO (ningún executeQuery en el main o en la GUI)
- ☐ El .zip incluye: todos los .java + el script .sql + instrucciones de ejecución

8.1 Descripción del escenario principal demostrado en el main()

Explica qué secuencia de acciones ejecuta tu main() para demostrar que el sistema funciona. Ej: 'Se registra un producto, se vende a un cliente, se verifica el stock y se muestra el historial de ventas.'

8.2 Capturas de pantalla de la interfaz

Inserta capturas de la interfaz gráfica mostrando al menos 3 pantallas distintas del sistema.